Generative AI System Design Interview
02 Gmail Smart Compose

1 Introduction and Overview
2 Ad Click Prediction
3 Video Search
4 Google Search
5 ChatGPT Personal Assistant
6 Image Captioning
7 Machine Translation
8 Recommendation System
9 Fraud Detection
10 High Resolution Image Synthesis
11 Text to Image Generation
12 Text-to-Video Generation
13 Text-to-Video Generation

02 Gmail Smart Compose

Introduction

Gmail's Smart Compose feature [1] saves users by suggesting the next few words as they write an email. This chapter explores the theory and architecture behind this production-scale generative AI system.

Figure 1: Gmail's Smart Compose feature

Clarifying Requirements

Here is a typical interaction between a candidate and a interviewer.

Candidate: Does the suggestion need to be personalized, or should it be the same suggestion for all users?

Interviewer: For simplicity, let's assume it's not personalized.

Candidate: Should the system suggest the next few words, or only when it is confident in its prediction?

Interviewer: Yes.

Candidate: The small dataset must be sufficiently large to train a model. Do we know the approximate size of the data?

Interviewer: Assume our dataset consists of around one million sentences.

Candidate: There are different parts of speech in a sentence where making suggestions for example, a pronoun is well-suited to a verb but not an adjective. Should the model be aware of the grammar of the sentence?

Interviewer: Good point. In practice, it does matter, but for the sake of time, let's assume that the user has typed in the current word, and it ends at the trailing of the period. If we have more time, we can expand the context to include other relevant information.

Candidate: What metrics do we need to track for evaluation?

Interviewer: Let's focus on the latency of the model.

Interviewer: That is an important requirement for the system. The system should not make biased suggestions or prohibitively by suggestions.

Candidate: How many words users does Gmail have? Is the computing cost a concern in this feature?

Interviewer: Gmail has about 10 billion users, and a single user can send as many emails in a day. We do care about the cost, but let's focus on the latency requirement for the system for the sake of time.

Candidate: Should the system suggest the same suggestion for all users, or should it be personalized?

Interviewer: Yes. The suggestions should be personalized. For example, suggesting around 100 individuals should be the same.

Frame the Problem as an ML Task

In this section, we frame the Smart Compose feature as an ML task. The section requires an understand the system's inputs and outputs and choose a suitable ML paradigm to solve the problem.

Specifying the system's input and output

The input to the model is a sequence of words typed to the user. The output is a continuation of that sequence. The model generates the suggestion if it is likely to type next.

Figure 2: Input and output of the Smart Compose system

Choosing a suitable ML approach

Smart Compose generates textual content, so we categorize it as a text generation task. Various ML architectures are designed to process sequential data, which is essential for text generation. The popular architectures are recurrent neural networks (RNN) and Transformers [2].

Transformers provide several advantages over RNNs, with two main benefits being:

- **Parallelism:** In an RNN, computations from one time step are carried forward and used in the next, creating a time dependent chain that hinders parallelism. In a Transformer, the attention layer processes all tokens simultaneously, boosting computational efficiency.
- **Better handling of long sequences:** Transformers can with attention mechanisms to focus on any part of a sequence, regardless of its length. In contrast, RNNs struggle with long sequences due to the vanishing gradient problem and their sequential nature.

Due to these advantages, Transformers have shown outstanding performance in text generation tasks and are, thus, used in most generative sequence modeling. Therefore, we choose Transformers for the Smart Compose system.

	Recurrent Neural Network (RNN)	Transformer
Architecture	Recurrent	Convolutional
Training efficiency	Inefficient due to sequential processing	Efficient due to parallel processing
Long-range dependencies	Struggles with long-range dependencies	High at the attention layer
Scalability	Limited scalability	Highly scalable
Applications	Simple tasks such as time series forecasting	Complex tasks such as language completion or translation

Figure 3: Comparison of RNN and Transformer architectures

While Transformers are more predominant due to their advantages over RNN, they still have significant drawbacks. Feed-forward neural networks (FFNN) must be added to the output of the attention layer to introduce non-linearity. Additionally, multiple recurrent layers can be stacked in RNNs to create complex sequences, but adding more layers to Transformers can be introduced to reduce the complexity of attention. To learn more, refer to Group Attention [3] and Flash Attention [4].

Data Preparation

During this data preparation step, we convert raw data into the format supported by the ML model. First, let's briefly review the available data.

The sources of data are available for training our model general data and email data. General data includes publicly available text data such as books, articles, and web pages. The model learns the English language and grammar from this data. In contrast, email data includes emails from Gmail.

Figure 4: Two sources of data for the Smart Compose system

The email data, as specified in the requirements, consists of one billion email messages. This data is crucial for the model to learn the specific patterns and nuances of email communication, such as the structure of greetings, the tone of formal versus informal emails, and common phrases in business and personal emails.

Dataset	Sender	Recipient	Subject	Body
email1	jsmith@gmail.com	mjones@gmail.com	Lunch	Hey Mike, Want to meet for lun...
email2	mwilliams@gmail.com	ed@hotmail.com	Project Deadline	Hi Ed, I hope you are well. I am writing to you to ...

Figure 5: Example of email data

Next, let's look general data and what it contains. It includes various types of content, which can impede the model performance. Additionally, the massive volume of data is not in a substantial format. For these reasons, we then try to prepared using the following two key steps:

- Text cleaning and normalization
- Text tokenization and token indexing

Text cleaning and normalization

Text cleaning

Text cleaning transforms unstructured or irrelevant information. Common techniques include:

- **Removing HTML tags:** Web pages often contain HTML tags that add structure but don't directly add remove non-English text from general and email data.
- **Remove confidential information:** Emails may contain confidential information such as phone and credit card numbers. To prevent the model from learning and potentially leaking this sensitive data, we can remove or anonymize it. Common patterns include email addresses, and phone numbers. For example, replace with placeholders like "email@example.com" or "[redacted]".
- **Handle special characters or symbols:** Remove or convert inconsequential characters and symbols that do not contribute to the meaning. Examples include emoji and non-ASCII or special characters, depending on the model's intended use.
- **Remove duplicate data:** Duplicate data refers to identical text from different sources that appear multiple times in the dataset. We remove duplicates to prevent the model from learning these sequences and skewing the model's training process.

Text normalization

Text normalization transforms text into a consistent format. For example, it converts different ways of writing a phrase number — "one," "1," and "first" — into a single, standard format. This ensures the model treats these variations as the same concept, reducing vocabulary size and enabling more efficient learning.

Here, we convert the raw text into a sequence of numbers through text tokenization and token indexing.

Text tokenization and token indexing

Text tokenization breaks down raw text into smaller units called tokens. Figure 6 shows how OpenAI's GPT-3 tokenizes the sentence "Let's go for a walk!" [5].

Figure 6: Example of GPT-3 tokenization

Tokenization can be performed at different levels. For example, "Let's go!" can be tokenized into "Let", "'s", "go", "!" (word-level), "L", "e", "t", "'", "s", "g", "o", "!" (character-level), or "Let", "'s", "go", "!" (subword-level). Each level offers different trade-offs:

- Character-level tokenization
- Word-level tokenization
- Subword-level tokenization

Character-level tokenization creates small vocabularies but generates lengthy sequences that slow model processing. Word-level tokenization creates long sequences but requires large vocabularies to capture all words in a language. Subword-level tokenization balances these approaches, breaking down rare words into meaningful subunits while keeping common words intact. This approach maintains reasonable vocabulary sizes and ensures the model can handle unfamiliar words by breaking them into known subwords.

Figure 7: Example of character-level tokenization

Figure 8: Example of word-level tokenization

Figure 9: Example of subword-level tokenization

While subword-level tokenization can be complex to implement, it has several benefits. First, it adapts to a manageable vocabulary size while still handling rare words effectively by breaking them into meaningful subunits. This approach also allows the model to represent word roots, suffixes, and prefixes, potentially improving its understanding of word meanings.

Table 2 lists common characteristics of the three tokenization categories.

Characteristic	Character-level	Word-level	Subword-level
Granularity	Individual characters	Whole words	Subwords
Vocabulary size	Small	Large	Medium
Handle unknown words	Simple	Simple	Complex
Handle spelling errors	Decomposes unknown words into characters	Cannot easily handle unknown words	Decomposes unknown words into known subwords
Vocabulary size	100	100,000+	10,000-50,000
Performance	Lengthy sequences hinder NN performance	High-performance but very memory-intensive	High-performance and efficient

Table 2: Comparison between different tokenization levels

Which tokenization is suitable for the Smart Compose feature?

Most state-of-the-art large language models (LLM) use subword-level tokenization algorithms such as Byte Pair Encoding (BPE) [10] and WordPiece [11]. These tokenization algorithms are more efficient and can effectively handle multiple languages. For example, OpenAI's GPT-4 uses a variant of BPE, and Google's BERT uses a variant of WordPiece [12].

Given the effectiveness of subword-level tokenization, we use it for our tokenizer for the Smart Compose feature. We can, or we can train a tokenizer on our data using libraries like Google's SentencePiece [13] or OpenAI's tiktoken [14]. Training a custom tokenizer allows us to control the vocabulary size and ensure optimal tokenization for our specific data.

In Chapter 5, we will dive into BPE and WordPiece in depth algorithms. To learn more about subword-level tokenization algorithms, refer to [15].

Token indexing

Token indexing is the process of converting textual tokens into integer numbers.

To prepare for token indexing, the tokenization algorithm first builds a vocabulary—a collection of all unique tokens from the training data and assigns each token a unique integer. Next, it scans a corpus of sentences, breaking each sentence into tokens. The order and values of tokens are stored sequentially for demonstration purposes.

Figure 10: Example of token indexing

To summarize the data preparation step, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 11: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 12: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 13: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 14: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 15: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 16: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 17: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 18: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 19: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 20: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 21: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 22: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 23: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 24: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 25: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 26: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 27: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 28: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 29: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 30: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 31: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 32: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Compose feature, the model must understand the sequence of words already entered by the user and suggest the next words.

Transformers have three primary components:

- Encoder only
- Decoder only
- Encoder decoder

Each component has unique architectural differences that make them suitable for specific tasks. Let's briefly explore each variation and its applications.

Encoder-only

An encoder-only Transformer is used for tasks that require understanding the overall meaning of a text. It processes the input sequence bidirectionally, allowing each token to attend to all other tokens in the sequence. In a sentiment analysis task, an encoder-only Transformer predicts the sentiment of the input sentence.

Figure 33: Encoder-only Transformer for sentiment analysis

Encoder-only Transformers are commonly used for tasks such as sentence classification and named entity recognition, which leverage bidirectional context to understand the relationships between words in a sequence. Popular encoder-only models include the family of BERT variants. Transformer encoders can also be used for embeddings, used to generate rich sequences. Discussed later, Transformers, on the other, are specifically designed for that purpose.

Decoder-only

A decoder-only Transformer processes the input sequence and generates a new sequence iteratively.

Figure 34: Decoder-only Transformer for next completion

To prepare for token indexing, we first clean and normalize the raw data to ensure high-quality, consistent text in our training data. Next, we use a subword-level tokenization algorithm such as BPE or WordPiece to break down text into manageable units and then replace each token with its numerical index. These steps ensure our training data is now represented in a numerical format ready for model processing.

Model Development

The Smart Compose feature is not production-level in which a transformer model predicts next words/phrases are likely to completed. In this section, we explore the details of the transformer architecture, training strategies, and sampling methods to develop the text generation model.

Architecture

The transformer architecture, introduced in the paper "Attention Is All You Need" [2], is designed to process sequences. This makes it ideal for tasks that require understanding in text and the relationships between its words. For example, in the Smart Comp

embeddings so they can be added together (see Figure 5).

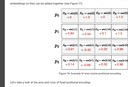

Figure 15: Example of sine curve positional encoding

Let's take a look at the pros and cons of fixed positional encoding.

Pros:

- **Efficiency:** Fixed encodings do not add extra trainable parameters to the model. This makes them computationally less intensive.
- **Consistent Representation:** Fixed encodings use a set pattern to represent each position. This flexibility allows the model to handle longer sequences without the need for retraining.

Cons:

- **Inflexibility:** Since fixed encodings are set, they don't adapt to the specific tasks or datasets. This limits their capability to capture more nuanced or complex patterns in the data.

Let's take a look at learned positional encoding.

Learned positional encoding has the following pros and cons.

Pros:

- **Optimal performance:** Since the embeddings are trained based on the training data, learned positional encoding can lead to more optimized representations for the specific task.

Cons:

- **Inefficiency:** Requires additional parameters to be learned during the training, which can increase the training time and computational cost.
- **Lack of generalization:** Learned embeddings may overfit to specific sequence lengths seen during training. If the model encounters sequences of different lengths during inference, it might struggle with generalizing to unseen sequence lengths.
- **Sensitivity to initialization:** The quality of learned positional encodings can be sensitive to the initialization.

In summary, the choice between learned and fixed positional encodings depends on the constraints of the task, including the available computational resources and the need for generalization. Fixed encodings offer efficiency and consistency, learned encodings aim for efficiency and flexibility but risk inflexibility. Following that, we analyze fixed positional encoding, such as sine curve encoding, to train the Smart Compose feature.

Transformer

The Transformer component takes a sequence of embeddings as input and transforms them into an updated sequence of embeddings.

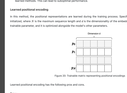

Figure 16 Embedding Transformer block

The Transformer architecture consists of a stack of identical blocks. Each block contains two sublayers:

- **Multi-head attention:** This layer applies self-attention to capture long-range dependencies. The attention mechanism calculates relevance between different parts of the sequence by allowing each embedding to attend to other embeddings in the sequence. This helps the model understand context and relationships between words or subwords, even if they are far apart in the sequence.
- **Feed-forward network:** This layer applies non-linear transformations to each embedding independently. It adds complexity to the model and allows it to learn more abstract representations.

Transformer architecture includes details such as residual connections, layer normalization, and dropout layers. For a deep understanding of these components, refer to the Transformer tutorial on PyTorch. We'll skip them here.

Prediction head

The prediction heads offer final components for a decoder-only Transformer. Contrastive the Transformer's output, these predictions for every token in the vocabulary (Figure 17). The model uses these predictions to choose the next likely token.

Figure 17: Prediction head output probabilities

Training

Training defines the decoder-only Transformer's parameters using small steps. Once the training process is complete, the model uses these parameters for predictions.

However, efficiently training the model on a vast corpus presents several challenges. We address these challenges using techniques like:

- **Lack of large training data:** Task-specific datasets are usually limited in size. This limitation can hinder the model's ability to learn robust patterns.
- **Risk of overfitting:** When a model is trained on a small specific dataset, it can edge out of generalizing to unseen data. Overfitting occurs when a model memorizes the training data to the extent that it cannot generalize to unseen data.
- **Computational cost:** Training a language model, even a decoder-only Transformer, demands significant computational resources and time. Factors like the model's size and the chosen optimization algorithm impact the overall cost of training.

To address the above issues, a two-stage training strategy is commonly employed: pretraining followed by fine-tuning. In the pretraining phase, the model learns general-purpose language representations by predicting the next token in a large dataset of text. This pretraining helps the model develop a strong understanding of language. Fine-tuning then adapts the pre-trained model to a specific task using a smaller dataset relevant to the target task. This process leverages the knowledge gained during pretraining and refines the model's parameters for optimal performance on the specific task.

This two-stage strategy benefits from a lack of monitor learning, as the general knowledge gained during the pretraining stage is transferred to the fine-tuning stage. This transfer is beneficial because the model doesn't need to learn from scratch when learning a new task. Instead, it adjusts its pre-trained weights, which is more efficient.

Figure 18: Two stage training strategy

Let's take a closer look at each stage and examine the necessary training tasks. We objectives, and loss functions for each.

1. Pretraining

Pretraining involves training a model on a vast dataset of text. The goal here is to develop a broad understanding of language. The model learns general language patterns, grammar, and even some factual knowledge. This pre-trained model then serves as a foundation for further fine-tuning on specific tasks, leveraging its acquired linguistic capabilities.

Pretraining data

The pretraining data for this stage usually consists of a large corpus of general text data from various sources on the web, such as web pages, books, and social media. For example, Common Crawl (CC) is a publicly available dataset collected by crawling a large number of web pages. A recent English language variant of Common Crawl has been publicly collected since 2020.

Objective and loss functions

In NLP, objective refers to the mathematical goal that a training process aims to achieve. In the case of next generation, the most commonly used NL objective is "next token prediction." In this NL objective, the model is tasked with predicting the next token (or word) in a sentence based on the preceding tokens. For example, if the sentence "I have a pet cat. The model should predict a high probability for "cat" as the next token.

Figure 19: Unsupervised generation of LM

To optimize the model for correctly predicting the next token, we define a loss function to guide the training process. Cross-entropy loss (CE) is a commonly used loss function in conjunction with next token prediction. This loss function measures the differences between the predicted probabilities and the correct token. This loss allows the optimizer to update the model's parameters to minimize future prediction errors.

Figure 20: Loss calculation

In practice, the model processes all token lengths within a sequence in parallel. This allows it to compute the loss for each token position simultaneously. Parallelizing this end-to-end process accelerates the training process instead of sequentially.

Figure 21: Parallelizing loss computations over different lengths

2. Fine-tuning

Fine-tuning involves adapting the base model from the pretraining stage to a specific task such as email completion. This stage focuses on training the model parameters to predict text by training on a smaller, task-specific dataset. Fine-tuning The model leverages the broad language understanding from the pretraining stage but adapts to the nuances of the task.

Fine-tuning data

We use a dataset of approximately one million email conversations, as specified in the requirements section. This data includes various email threads, both personal and business, and specific occurrences like meeting requests and product inquiries.

Objective and loss function

The objective for fine-tuning remains the next token prediction and cross-entropy loss concept. However, the focus shifts to the task-specific data. For instance, with email completion, the model learns to predict the next token in a sentence within the context of an ongoing email thread.

Figure 22: Fine-tuning task and loss calculation

It practice, the model processes all token lengths within a sequence in parallel. This allows it to compute the loss for each token position simultaneously. Parallelizing this end-to-end process accelerates the training process instead of sequentially.

Figure 23: Parallelizing loss computations over different lengths

3. Evaluating

Evaluating involves assigning the base model from the pretraining stage to a specific task such as email completion. This stage focuses on making the model proficient in a particular task by training on a smaller, task-specific dataset. Evaluating The model leverages the broad language understanding from the pretraining stage but adapts to the nuances of the task.

Evaluating data

We use a dataset of approximately one million email conversations, as specified in the requirements section. This data includes various email threads, both personal and business, and specific occurrences like meeting requests and product inquiries.

Objective and loss function

The objective for evaluating remains the next token prediction and cross-entropy concept. The NL objective is next-token prediction, and the cross-entropy loss function guides the training process. The only difference from the pretraining stage is that the loss is calculated based on email data, focusing on predicting the next token in an email context.

Figure 24: Fine-tuning task and loss calculation

It practice, the model processes all token lengths within a sequence in parallel. This allows it to compute the loss for each token position simultaneously. Parallelizing this end-to-end process accelerates the training process instead of sequentially.

Figure 25: Parallelizing loss computations over different lengths

Combining various inputs

A traditional NL NL model typically processes inputs of the same type of data in processes. This requires customized preprocessing and feature engineering for different data types like text, images, or tables.

In the use of Smart Compose, the model architecture is often discussed from the input structure. This decomposing increases the model's versatility and ability to learn from various data types. The model can then leverage shared layers and attention mechanisms and embeddings for beneficial of Smart Compose. This flexibility in data format accelerates the primary engineering (DE), in Chapter 4 we examine primary engineering in detail.

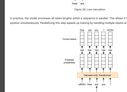

Figure 26: Traditional ML vs Prompt Engineering

The benefits of two-stage training

The two-stage training strategy has several benefits, including:

- **Adaptability:** The same base model pretrained from the pretraining stage can be adapted for different tasks.
- **Improved generalization:** Pretraining on large and diverse text data enables the model to develop a broad understanding of language. This understanding generalizes well to downstream tasks.
- **Fast Pretraining:** The model learns general knowledge during the pretraining stage, which speeds up the fine-tuning process. This allows for quick adaptation to new tasks.
- **Resource efficiency:** Fine-tuning a pre-trained model requires less computational resources than training a model from scratch. This makes it more cost-effective and accessible.
- **Mitigating overfitting:** If we train a model from scratch on a smaller, task-specific dataset, there is a risk it will overfit. It has the capacity to memorize the training data, which hurts its ability to generalize to unseen data. Pretraining helps mitigate this by providing a robust initialization.
- **Resource optimization:** By separating the training process into two stages, resources can be allocated efficiently. The pretraining stage can be performed on a large scale by organizations with ample resources, while fine-tuning can be done on smaller, more accessible hardware. This reduces computational costs and makes it more feasible for fine-tuning models for each task.

Sampling

Generated sequences of tokens are created by the underlying distribution of the training data. Once trained, these models can generate text by sampling from the learned distribution. For example, if the model has been trained on a dataset of writing in a trained grammatically correct sentences, it will generate new text.

In the context of Smart Compose, sampling involves generating a short email completion given the user's partial email body and subject. The model might suggest completions like "I'm attaching the requested documents" or "Let me know if you have any questions." The goal is to leverage the model's learned patterns to generate relevant and helpful suggestions that assist the user in composing their email. Yet, does, does sampling work? Is provided to the model as input, and the "next" token is chosen as the next token. The process continues until the model reaches the EOS token.

Input
Model
Sampling
Output
Input
Model
Sampling
Output
Input
Model
Sampling
Output

In this equation:

- Σ is a tokenized sequence $\{x_1, x_2, \cdots, x_N\}$ in the test data that is used to evaluate how accurately the model predicts the sequence.
- N is the number of tokens in the sequence.
- $P(x_i \mid x_{0:i-1})$ is the conditional probability of the i-th token given the preceding tokens, $x_{0:i-1}$, that is, how likely the model is to predict the i-th token given the previous tokens.

Figure 38: Illustrates a concrete example to better understand perplexity.

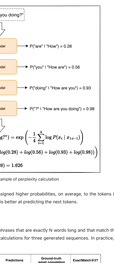

Figure 39: Example of perplexity calculation

A lower perplexity value indicates that the model has assigned higher probabilities, on average, to the tokens that appear in the test data. Therefore, a lower perplexity means the model is better at predicting the next tokens.

ExactMatch@N

ExactMatch@N measures the percentage of generated phrases that are exactly N words long and that match the first N words of the ground-truth text. Figure 39 shows ExactMatch@3 calculations for three generated sequences. In practice, there are usually more than three sequences to evaluate.

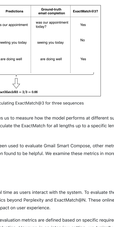

Figure 39: Example of calculating ExactMatch@3 for three sequences

Calculating ExactMatch@N for different values of N allows us to measure how the model performs at different suggestion lengths. To measure the overall performance of the model, we calculate the ExactMatch for all lengths up to a specific length and then take the average.

While Perplexity and ExactMatch@N have traditionally been used to evaluate Email Smart Compose, other metrics such as BLEU score and ROUGE-N, introduced more recently, have been used to evaluate the language model. We examine these metrics in more detail in Chapter 3.

Online evaluation metrics

Online evaluation measures how a model performs in real time as users interact with the system. To evaluate the Smart Compose feature in an online environment, we use additional metrics beyond Perplexity and ExactMatch@N. These online metrics measure user engagement, the model's latency, and the overall impact on user experience.

Unlike offline metrics, which are usually standard, online evaluation metrics are defined based on specific requirements and needs. Companies often use hundreds of metrics for online evaluation. However, in an interview setting, we typically discuss the most common ones. In this section, we focus on the following metrics:

- User engagement metrics
- Effectiveness metrics
- Latency metrics
- Quality metrics

User engagement metrics

- **Acceptance rate:** The percentage of suggestions made by the Smart Compose feature that are accepted by users. A higher acceptance rate indicates that the suggestions are relevant and useful to users.
- **Usage rate:** The percentage of all composed emails that have utilized the Smart Compose feature. High usage rates typically indicate that users trust the feature.

Effectiveness metrics

- **Average completion time:** Tracks the average time taken by users to compose emails with and without the aid of Smart Compose. A reduced average completion time using Smart Compose will indicate that the feature is speeding up the email writing process.

Latency metrics

- **System response time:** Measures the time it takes for the Smart Compose suggestions to appear after the user begins typing. It's important to ensure this metric stays below a certain threshold so the suggestions are made before the user types them.

Quality metrics

- **Feedback rate:** Measures the rate at which users provide feedback on the suggestions. Feedback is helpful for continuous improvement of the system.
- **Human evaluation:** Qualitative assessments through user studies are employed to evaluate the usefulness of suggestions. This metric reflects user satisfaction with the Smart Compose feature.

These online metrics are essential for evaluating how well Smart Compose feature works in production. By monitoring these metrics, the stakeholders can obtain a holistic view of feature's performance.

Overall ML System Design

In this section, we propose a design for a Smart Compose system.

When designing such a feature, we should consider more than just the underlying model that predicts the next token. The system's effectiveness depends on various components working together to ensure the system is responsive, generates relevant suggestions, and maintains ethical standards. For the Smart Compose feature, we examine the following key components:

- Triggering service
- Phrase generator
- Post-processing service

Let's explore each in more detail.

Triggering service

The triggering service activates the Smart Compose feature by monitoring user activity such as keystrokes. It decides when to activate the feature based on criteria such as the number of characters typed or the entering of specific keywords in the text. For example, if a user types "I" the service might not activate Smart Compose because it's too early to predict the user's intent. However, if the user types "I hope," the service will activate Smart Compose, as the additional context allows for more useful suggestions.

The triggering service ensures suggestions are not too frequent. Once the service determines that activating the Smart Compose feature will be useful, it triggers the phrase generator component, which we discuss next.

Phrase generator

The phrase generator is the core of the Smart Compose feature. It generates the most likely completion based on the partial text the user has already typed.

To achieve this, the phrase generator interacts with the trained model and employs beam search to generate the top-k most probable completions. Each completion ends with the EOS token and an associated score that indicates how confident the model is about the completion.

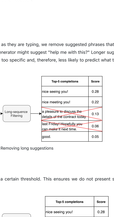

Figure 40: Beam search outputs top five potential completions (beam width = 5)

Given the possible completions, two critical considerations are necessary:

- Removing long suggestions
- Removing low-confidence suggestions

Removing long suggestions

As shorter suggestions are easier for the author to read as they are typing, we remove suggested phrases that are too long. For example, if a user types "Can you please" the phrase generator might suggest "help me with this?" Longer suggestions, such as "help me with this project that is due next week," will be too specific and, therefore, less likely to predict what the author intends to write.

Figure 41: Removing long suggestions

Removing low-confidence suggestions

We remove suggestions with confidence scores below a certain threshold. This ensures we do not present suggestions if the model is not confident enough about it.

Figure 42: Removing low-confidence suggestions

Finally, if the final list of suggestions is not empty, the phrase generator will pass the one with the highest confidence score to the post-processing service.

Post-processing service

The post-processing service addresses potential biases before suggestions are presented to the user. This component follows predefined rules to detect and correct bias efficiently. Common strategies to achieve this include:

- **Pronoun replacement:** Replace gender-specific pronouns to ensure neutrality. For example, "he" or "she" might be replaced with "they" in contexts where the gender is not specified.
- **Gender-neutral word replacement:** Replace gendered words with gender-neutral alternatives where appropriate. This includes changing words like "chairman" to "chairperson" or "policeman" to "police officer."
- **Lexical analysis for sensitive terms:** Use a predefined list of flagged terms that, if identified, can be replaced with neutral alternatives. For example, terms that might imply age, race, or disability biases are adjusted to ensure the suggestions will be perceived as respectful and neutral.
- **NSFW (Not Safe For Work) content filtering:** Implement automated filters that scan for and flag explicit language. These filters use predefined lists of NSFW keywords, phrases, and patterns to detect and remove problematic content.

By implementing these rules, the post-processing service maintains ethical standards in the Smart Compose feature, thus ensuring that the suggestions provided are relevant, respectful, and inclusive.

Figure 43: Smart Compose feature overall design

Here is a brief step-by-step workflow of the overall ML system employed by the Smart Compose feature:

1. **Monitoring:** The triggering service monitors the user's activity as they type.
2. **Triggering:** The service triggers the phrase generator once it identifies specific patterns.
3. **Beam search:** The phrase generator employs beam search to get top-k potential completions from the trained model.
4. **Filtering:** The phrase generator interacts with the filtering component to remove long suggestions and those with low confidence scores.
5. **Post-processing:** The completion with the highest score is picked and passed to the post-processing service. The service applies gender-specific pronouns and adjusts sensitive terms.
6. **Display suggestion:** The suggestion is displayed to the user for their consideration.

Other Talking Points

If there's extra time at the end of the interview, you may face follow-up questions or be asked to discuss advanced topics. This depends on factors such as the interviewer's preference, your expertise, and the requirements of the role. For senior roles, here are some topics you should prepare for:

- Supporting Smart Compose in multiple languages [26].
- Personalizing suggestions [28].
- Incorporating additional context for better predictions [29].
- Understanding how different tokenization algorithms work, such as BPE [11], SentencePiece [12], and WordPiece [29].
- Understanding different ML objectives such as masked language modeling (MLM) and its variations [18].
- The multi-token prediction objective and its pros and cons [30].
- Balancing quality and inference latency [28].

Summary

Reference Material

[1] Gmail's Smart Compose feature. https://research.google/blog/smart-compose-real-time-assisted-writing/.
[2] Fundamentals of Recurrent Neural Network. https://arxiv.org/abs/1808.03314
[3] Attention Is All You Need. https://arxiv.org/abs/1706.03762
[4] Gated recurrent unit. https://en.wikipedia.org/wiki/Gated_recurrent_unit.
[5] Long Short-Term Memory. https://deeplearning.cs.cmu.edu/F23/document/readings/LSTM.pdf.
[6] RTA: Group Attention is All You Need for Timeseries Analytics. https://arxiv.org/abs/2306.01929.
[7] Transformation: Fast and Memory-Efficient Exact Attention with IO-Awareness. https://arxiv.org/abs/2205.14135.
[8] Language Identification. https://en.wikipedia.org/wiki/Language_identification.
[9] FastText model for language identification. https://huggingface.co/facebook/fastext-language-identification.
[10] Transformer-XL. https://arxiv.org/abs/1901.02860
[11] Byte-Pair Encoding tokenization. https://huggingface.co/learn/nlp-course/en/chapter6/5.
[12] SentencePiece tokenization. https://arxiv.org/abs/1808.06226.
[13] Transformers. https://github.com/huggingface/transformers.
[14] Google's Gemini. https://gemini.google.com/.
[15] SentencePiece library. https://github.com/google/sentencepiece
[16] Summary of tokenizers. https://huggingface.co/docs/transformers/en/tokenizer_summary.
[17] OpenAI's tokenizers. https://tiktokenizer.vercel.app?model=gpt-4-1106-preview.
[18] BERT. https://arxiv.org/abs/1810.04805
[19] OpenAI's models. https://platform.openai.com/docs/models.
[20] Meta's LLaMA. https://llama.meta.com/.
[21] Introduction to Transformers by Andrej Karpathy. https://www.youtube.com/watch?v=XfpMkf4rD6E.
[22] Transformer visualized. https://daleonai.github.io/transformer/.
[23] Common Crawl. https://commoncrawl.org/.
[24] Cross-entropy. https://en.wikipedia.org/wiki/Cross-entropy.
[25] Prompt engineering. https://platform.openai.com/docs/guides/prompt-engineering.
[26] Beam search. https://en.wikipedia.org/wiki/Beam_search.
[27] Perplexity. https://en.wikipedia.org/wiki/Perplexity.
[28] Gmail Smart Compose: Real Time Assisted Writing. https://arxiv.org/abs/1906.00080.
[29] WordPiece tokenization. https://huggingface.co/learn/nlp-course/en/chapter6/6.
[30] Better & Faster Large Language Models via Multi-token Prediction. https://arxiv.org/abs/2404.19737.

Footnotes

1. Visit https://platform.openai.com/tokenizer to see examples of different tokenizers. ↩

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Jobs
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Ace

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.